

OMN

Team 04 Final Presentation

Robot Programing with ROS
Summer Semester 2026

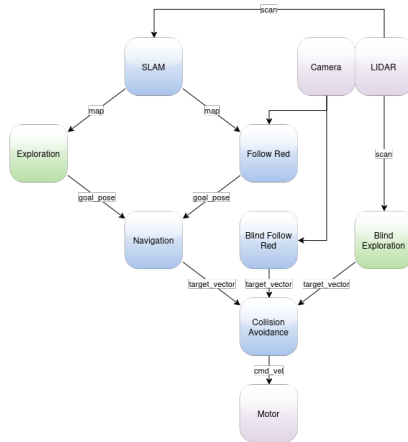
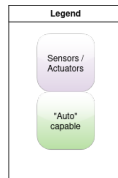
Jun 03, 2026

Outline

- 1 Overview
- 2 Exploration
- 3 Collision Avoidance
- 4 Navigation
- 5 Follow Red
- 6 Manager
- 7 Navigation (stuck, smoothing, θ^* v a^*)
- 8 Localization (in Sim)

Overview

Overview



Exploration

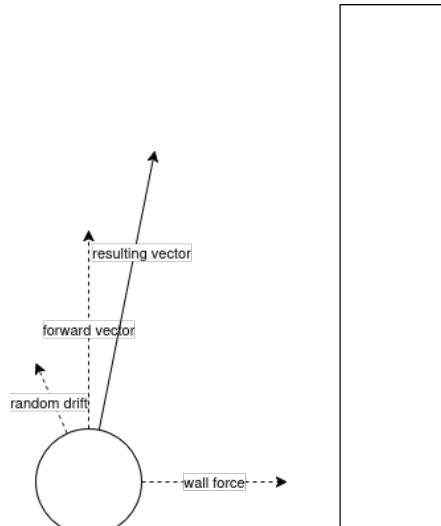
Blind Exploration

- No map given

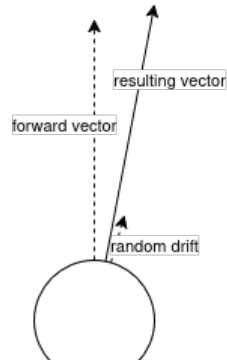
To complete the main goal "*Just keep moving!*" we do this:

- ➊ Apply random drift
- ➋ Drive forward
- ➌ Wall detected?
 - ➊ Yes: apply force to the side of the wall
- ➍ Haven't moved for a while?
 - ➊ Yes: apply randomized rotation

Blind Exploration



Blind Exploration - Why the drift?



A direction is picked through the random drift

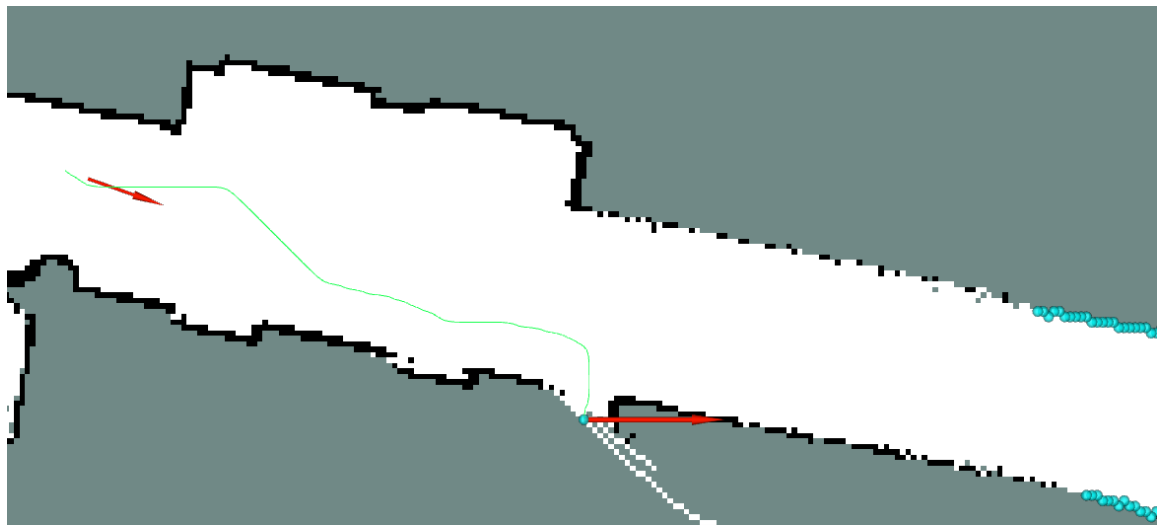
Exploration

- Sufficient exploration has taken place
- Map given

We now know where we are and where we've been, the goal changes to *"Explore the unknown!"*:

- ① Flood fill the map from our current position
- ② Create a frontier map
- ③ Select the closes frontier position (we haven't visited yet)
- ④ Send desired location to `/goal_pose`
- ⑤ Mark position as visited

Exploration

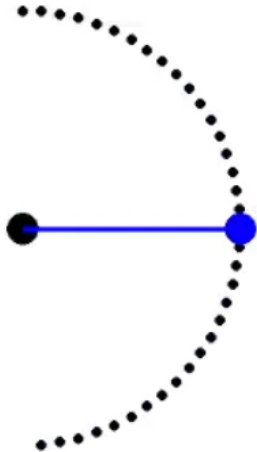


Collision Avoidance

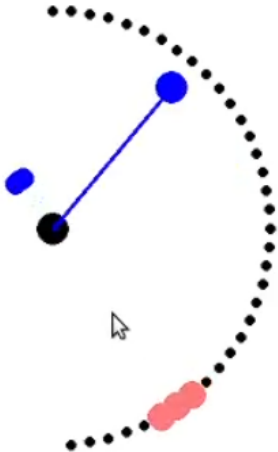
Collision Avoidance

- Has to be active if the robot wants to move
- Receives an `/target_vector` with a linear and angular velocity
- Ensures...
 - that the velocities are within safe bounds
 - that the movement direction does not point into obstacles
- Converts `/target_vector` to `/cmd_vel` and sends it off to motor

Collision Avoidance



Collision Avoidance



Navigation

Navigation Steps:

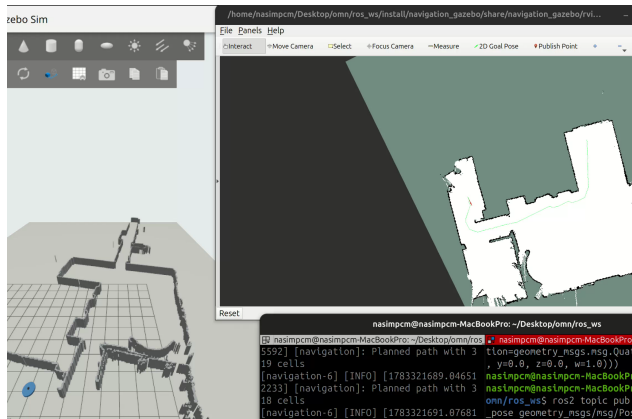
Planning

- 1 `timer_callback()` waits until a map, pose, and goal are available.
- 2 Robot and goal positions are converted from metres to map-grid cells.
- 3 A* plans a path using horizontal, vertical, and diagonal moves.
- 4 Obstacles are inflated to maintain clearance, and the grid path is smoothed to reduce zig-zag motion.

Path Following

- 5 Every 0.2 s, a waypoint approximately 0.4 m ahead is selected.
- 6 A `TargetVector` pointing toward the waypoint is published.
- 7 When the goal is within 0.4 m, the robot is stopped and `goal_succeeded` is reported.

A* Path Planning



Evaluation Function

$$f(n) = g(n) + h(n)$$

$g(n)$ is the path cost from the start and $h(n)$ estimates the remaining cost to the goal.

- Eight-connected grid search
- Straight-move cost: 1
- Diagonal-move cost: $\sqrt{2}$

A* Implementation

- ➊ **Initialize:** add the start cell to the open-set priority queue, set $g(\text{start}) = 0$, and initialize `came_from`, `g_score`, and the closed set.
- ➋ **Select:** remove the cell with the smallest estimated total cost $f(n)$.
- ➌ **Check the goal:** if selected, reconstruct the path from the stored parent cells.
- ➍ **Mark explored:** skip cells already processed; otherwise add the cell to the closed set.
- ➎ **Generate neighbors:** consider all eight adjacent cells using straight and diagonal move costs.
- ➏ **Apply obstacle costs:** reject blocked or invalid cells and penalize inflated cells near obstacles.
- ➐ **Update better paths:** if a lower cost is found, update the neighbor's parent and scores, then add it to the open set.
- ➑ **Finish:** return the reconstructed path, or report `no_path` if the open set becomes empty.

Next Waypoint Selection

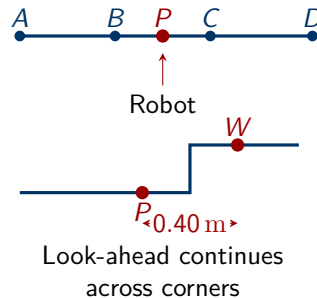
1. Locate the Robot on the Path

Project the robot onto every remaining path segment and select the closest point P .

2. Look Ahead Along the Path

Starting at P , measure 0.40 m along the path to obtain waypoint W . If the distance crosses a corner, continue on the next segment.

- `path_progress` prevents the selected position from jumping backward when the path passes close to itself.
- If less than 0.40 m remains, return the final goal point



Follow Red

Goal in 3 Steps

- ① Detect a red cube via OpenCV
 - ② Estimate its distance/angle or global map position
 - ③ Publish targets in a ROS 2 system
- **Node Type:** ROS 2 Lifecycle Node
 - **Input:** OpenCV camera stream
 - **Processing:** HSV masking, exponential smoothing, median filtering
 - **Output:** Polar coordinates (TargetVector) or global map poses (PoseStamped)

Given Hardware and Calibration

Type	Webcam
Frame	640x480
FOV	60
Calibrated Distance	1 Meter
Calibrated Pixels	9750

Dual Target Publishing Modes

Controlled by the runtime parameter `send_polar`:

Polar Mode (`send_polar = True`)

- Publishes `TargetVector` to `/target_vector`.
- Direct values for linear (distance) and angular (bearing angle).

Global Pose Mode (`send_polar = False`)

- Uses `tf2_ros` for transform lookups (`map` $\rightarrow$ `base_footprint`).
- Transforms camera relative coordinates into global map coordinates.
- Publishes `PoseStamped` to `/goal_pose`.

Image Processing & Filter Pipeline

1 Filter Size:

- Accept only a sum over 500px as possible cube.

2 Area Change Rate Filter (Jump Filter):

- Ignores frames with sudden, extreme area size spikes ($\frac{A_{new}}{A_{old}} \notin [\min, \max]$).
- Prevents position jumps caused by brief reflections or partial occlusions.

3 HSV Masking:

- Dual range masking for red hue due to HSV wrap-around ($0^\circ - 5^\circ$ and $174^\circ - 179^\circ$).

4 Low-Pass Filter (Exponential Smoothing):

- Noise reduction for pixel position ($\alpha_{pos} = 0.25$) and area ($\alpha_{area} = 0.20$).

5 Median Ring Buffer:

- `deque(maxlen=5)` eliminates sporadic false detections over 5 frames.

6 Distance Estimation:

- Inverse square-root calculation based on calibrated pixel area:

$$d = d_0 \cdot \sqrt{\frac{A_0}{A}}$$

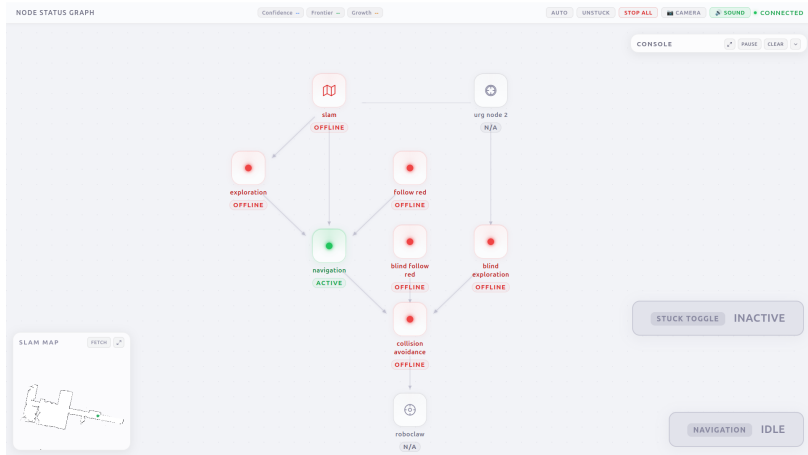
Fault Tolerance

Automatic Camera Reconnect

- Monitors consecutive read errors
- When reaching CAMERA_MAX_ERRORS (40 frames $\approx$ 2 s), the video device is released and re-initialized.

Manager

Dashboard



Robot Manager – Functionality Deep Dive

Health & Fault Detection

Every vital system is timed. Silence past a threshold is treated as failure – automatically.

1 – LiDAR /scan

- 0 s: healthy
- 1.5 s: **WARN**
- 4 s: **DOWN** – zero speed forced, task cancelled, state → FAULT

2 – Localization /pose

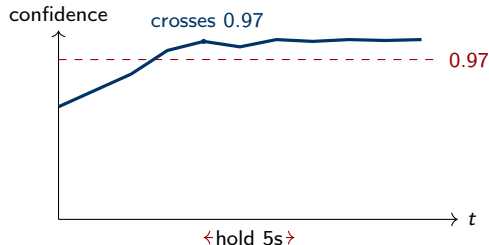
- 0 s: healthy
- 3 s: **WARN**
- 8 s: **DOWN** – same recovery as above

Why thresholds instead of instant failure?

A single missed message doesn't mean a sensor is broken – networks and drivers hiccup. Two thresholds (WARN, then DOWN) give a fair grace window before anything drastic happens, while still guaranteeing a fast, automatic stop once it's truly gone. Healthy again → back to INITIALIZING (re-verified).

Knowing When Mapping Is Actually Done

Confidence must stay above threshold for a sustained hold time – a single lucky spike doesn't count.



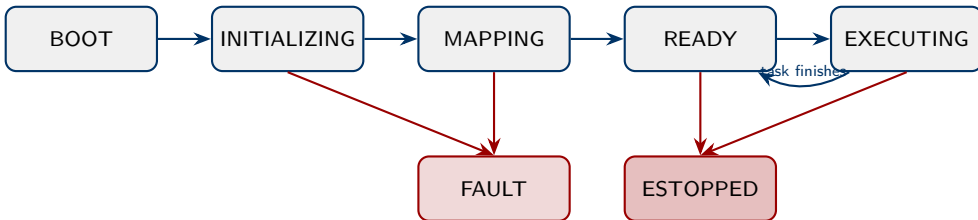
The hold-timer rule

- 1 Confidence first crosses 0.97 at $\approx 8\text{ s}$ → a hold-timer starts.
- 2 It must stay ≥ 0.97 continuously for 5 more seconds.
- 3 Only then is the map declared converged and saved.

If it dips below 0.97 even once, the timer resets – the wait starts fresh.

How a Finite State Machine (FSM) Runs the Mission

The robot is always in exactly one state, and a fixed rule table decides every legal transition.



FAULT / ESTOPPED are reachable from ANY state, instantly.

How it's implemented

A lookup table maps (current state, event) $\rightarrow$ next state. An event that isn't a valid key for the current state is simply ignored – illegal transitions are structurally impossible, not just unlikely – and every transition is logged with a timestamp.

Resolving Conflicts Automatically – and How

Two programs trying to drive the robot at once is unsafe. Robot Manager decides who wins, asymmetrically.

A – Starting an autonomous task

- 1 Operator clicks Explore / Follow
- 2 Manual driving mode is stopped automatically
- 3 New autonomous pipeline starts cleanly

Automatic hand-off – a background convenience process is low-risk to interrupt.

B – Interrupting an active task

- 1 Operator tries to switch to manual driving
- 2 Request is REJECTED with a clear reason
- 3 Operator must explicitly cancel the task first

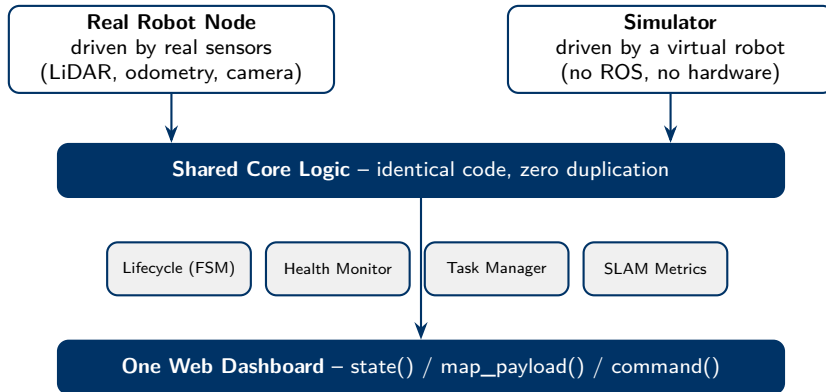
Deliberately NOT automatic – silently killing a mission the operator explicitly started would be surprising and risky mid-mission.

Deep Dive: Simulation Mode

A second, self-contained backend that runs the exact same brain against a virtual robot – built to test what the real robot never safely can.

Same Brain, Two Different Bodies

The decision-making code is never duplicated – only the world underneath it changes.



Both backends expose the exact same 3 functions, so the dashboard can't tell – and doesn't need to know – whether it's watching a real Tortugabot or a simulated one.

How the Virtual World Is Built

Four pieces of physics stand in for the real robot's sensors and drivers.

Map Source	Virtual Sensing	Path Planning	Virtual Target
Loads a real, previously-saved map if one exists. If none exists, it generates a synthetic maze arena instead.	A “reveal radius” around the simulated robot uncovers the map gradually as it moves.	Its own A* search with obstacle inflation – a self-contained stand-in for the real navigation planner.	A red cube that wanders the map, so the follow-cube task can be tested without a camera.

Runs standalone

A background thread ticks the virtual world 10 times a second – the same update rate the real robot uses – using only Python’s standard library. No ROS, no GPU, no extra installs: it runs on any laptop.

Fault Injection – Breaking the Robot on Purpose

The one test a real robot can never safely run: deliberately killing a sensor mid-mission to prove recovery works.

- ① Operator flips a toggle on the dashboard (e.g. “kill LiDAR”)
- ② That component's heartbeat is silently withheld – nothing else in the code path is touched or faked
- ③ Health Monitor ages it past WARN, then DOWN – the exact same logic used for a real failure
- ④ Lifecycle is driven into FAULT for real – zero velocity, task cancelled, state forced
- ⑤ Toggle released → heartbeat resumes → full recovery, back to INITIALIZING, re-verified from scratch

Why this proves something real

Nothing about the FAULT state itself is faked. Only the sensor's heartbeat is withheld – the Health Monitor, the FSM, and the recovery path are the real production code, discovering a real missing signal.

On the physical robot, testing this would mean unplugging a live sensor while it drives. In simulation it's one click, fully repeatable, and safe to run as many times as needed.

Simulator vs. Real Dashboard

Same product experience, different world underneath – and new abilities the real robot doesn't have.

Identical in both

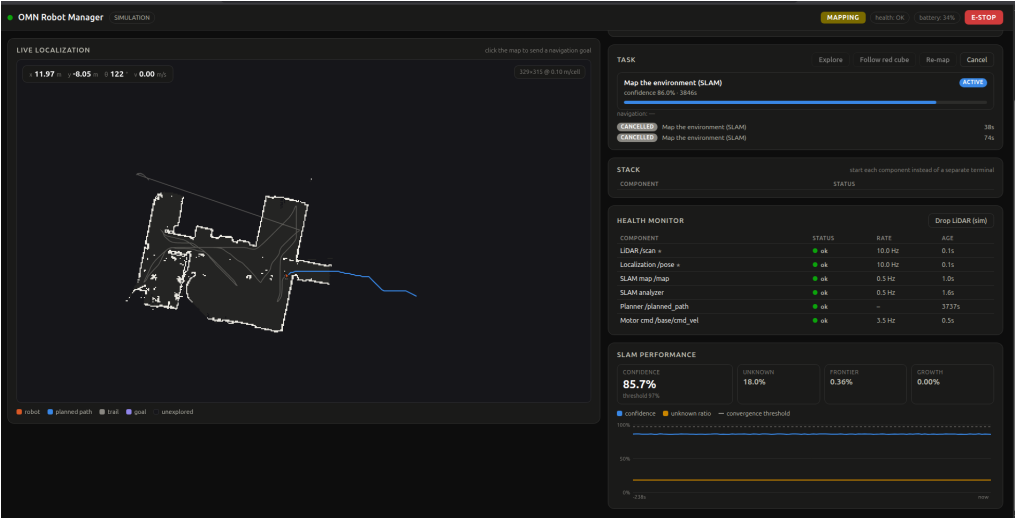
- The web dashboard UI, unchanged
- State machine and its transition rules
- Health thresholds and OK/WARN/DOWN logic
- “One task at a time” task rules
- Navigation goal tolerance & timeout values

Different in simulation

- Data comes from virtual physics, not real sensors
- Faults can be deliberately triggered on demand
- No lab access or physical robot required at all
- Runs anywhere – stdlib only, zero setup
- Generates its own map when none exists yet

Why it matters: the whole mission logic can be demoed, tested, and hardened for rare or risky scenarios – safely, repeatably, and without ever touching the physical robot.

Simulation Dashboard



Navigation (stuck, smoothing, θ^* v a^*)

Path Smoothing via LOS

LOS-Algorithm

- Path from A^* gets checked for shortcuts
- Line of Sight is used
- Starts from first point, checks for 50 cells ahead
- Looks at furthest away, moves closer until visible
- Repeat for the next point

Reasons for/against using

- **For:** Very smooth path
- **For:** Decreased number of waypoints
- **For:** Changeable Look-Ahead-Distance
- **Against:** High computational Cost
- **Against:** Too slow for constant path replanning
- **Result:** Not used

Path Smoothing via Gradient

Algorithm

- Path from A* gets corrected
- Multiple passes
- Each coordinate gets tied to its origin
- Gets pulled away by: $y_{next} + y_{prev} - 2.0 * y_{curr}$
- Stops automatically after specified passes or little change occurring

Reasons for/against using

- **For:** Low Computational Cost
- **For:** Tweakable
- **Against:** Path not as smooth
- **Against:** Might drag path through walls
- **Result:** Used

Theta*

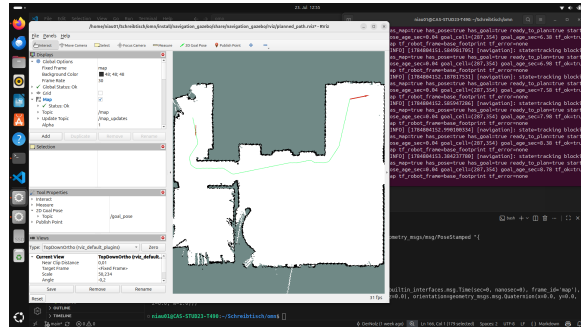
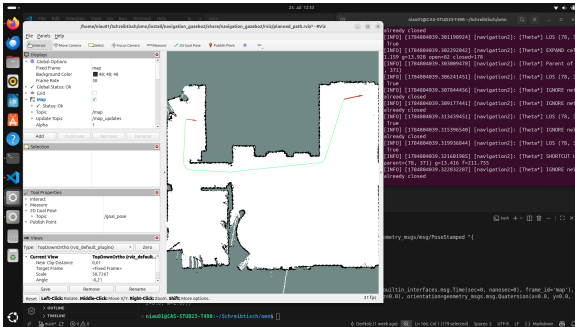
Essentially A* with shortcuts:

- Performs shortcut checks to find shorter path
- Checks the current parent for visibility of best neighbor
- If successful, skips current node and uses parent as parent of best neighbor
- Repeat

Theta* vs A*

Theta*

A*



Time of Computation: approx. 240 Seconds
Noticeably smoother path

Time of Computation: approx. 2 seconds
Rougher path even after path smoothing

==> A* used due to faster computation

Stuck Logic

Algorithm

- Only when in mode "tracking" to avoid timeout while calculating path
- Checks weather the robot has moved enough in a specified timeframe
- When stuck, clears the current path and publishes velocities = 0
- When next_goal in queue, switch over

Toggle Logic

Set toggle to false, when:

- goal is reached or new current goal detected
- stuck and there is a next goal
- path is cleared, there is no path or a new path was created
- not ready to plan (no pose, no map or no goal)
- replanning

Set toggle to true, when:

- following a path

Goal Queue: Problem and Solution

Previous Behaviour

- Navigation handled only one goal
- New goal replaced the active goal
- Older goals were discarded

New Behaviour

- Added FIFO goal queue
- Current goal stays active
- New goals wait in queue
- Goals follow received order

FIFO Order

Goal A $\rightarrow$ Goal B $\rightarrow$ Goal C

Goal Queue: Main Logic

When a New Goal Arrives

- ① Transform the goal into the map frame
- ② Check whether it is a duplicate
- ③ Activate it if no goal is active
- ④ Otherwise, append it to the queue
- ⑤ Reject it if the queue is full

Queue Settings

- Minimum spacing: 0.25 m
- Maximum size: 100 goals
- New goals added with:

`append()`

- Next goal taken with:

`popleft()`

Code Added In

- Import section and `__init__()`
- `goal_pose_callback()`
- New `activate_next_goal()` method
- Goal-reached part of `timer_callback()`

Goal Queue: Simulation Result

Waypoint Queued

```
navigation-7 [INFO] [178388859.835297231] [navigation]: Planned path with 132 cells; reason:robot off path
navigation-7 [INFO] [178388860.86236082] [navigation]: Got new position
navigation-7 [INFO] [178388866.367987728] [navigation]: Got new position
navigation-7 [INFO] [178388861.699728824] [navigation]: Got new position
navigation-7 [INFO] [178388861.934628883] [navigation]: Got new position
navigation-7 [INFO] [178388862.728160352] [navigation]: Planned path with 131 cells; reason:robot off path
navigation-7 [INFO] [178388862.739684281] [navigation]: Got new position
navigation-7 [INFO] [178388862.211593232] [navigation]: Got new position
rviz2-9 [INFO] [178388863.38456667] [rviz2]: Setting goal pose: Frame:map, Position(13.7475, -7.7469, 0), Orientation(0, 0, 0.596266, 0.8863369) = Angle: 2.9687
navigation-7 [INFO] [178388862.286028192] [navigation]: Queued goal x:13.747, y:-7.746; queue size=3
navigation-7 [INFO] [178388866.350111189] [navigation]: NAVIGATION STATUS | state=MOVING | goal=(5.17, -6.54) | queue=3 | Waypoint added to the navigation queue
navigation-7 [INFO] [178388863.877134339] [navigation]: NAVIGATION STATUS | state=MOVING | goal=(5.17, -6.54) | queue=3 | Following path to active waypoint
navigation-7 [INFO] [178388864.877862953] [navigation]: Got new position
navigation-7 [INFO] [178388864.576573798] [navigation]: Got new position
navigation-7 [INFO] [178388865.419148142] [navigation]: Got new position
navigation-7 [INFO] [178388864.965373239] [navigation]: Got new position
navigation-7 [INFO] [178388866.792473817] [navigation]: Planned path with 129 cells; reason:robot off path
navigation-7 [INFO] [178388866.798182481] [navigation]: Got new position
navigation-7 [INFO] [178388867.229925412] [navigation]: Got new position
navigation-7 [INFO] [178388867.832794193] [navigation]: Got new position
navigation-7 [INFO] [178388868.391624397] [navigation]: Got new position
navigation-7 [INFO] [178388869.267973276] [navigation]: Planned path with 128 cells; reason:robot off path
navigation-7 [INFO] [178388869.277062418] [navigation]: Got new position
navigation-7 [INFO] [178388869.720728178] [navigation]: Got new position
navigation-7 [INFO] [178388870.491743339] [navigation]: Got new position
navigation-7 [INFO] [178388871.212388128] [navigation]: Got new position
navigation-7 [INFO] [178388872.566481323] [navigation]: Planned path with 127 cells; reason:robot off path
```

- Queue size reached 3
- Active goal was not replaced

Queue Completed

```
navigation-7 [INFO] [178388879.564889886] [navigation]: Planned path with 4 cells; reason:robot off path
navigation-7 [INFO] [178388879.511888416] [navigation]: Got new position
navigation-7 [INFO] [178388872.488161230] [navigation]: Got new position
navigation-7 [INFO] [178388872.643636670] [navigation]: Planned path with 3 cells; reason:robot off path
navigation-7 [INFO] [178388872.644618162] [navigation]: Got new position
navigation-7 [INFO] [178388872.28178181] [navigation]: Got new position
navigation-7 [INFO] [178388872.737215585] [navigation]: Waypoint reached x:13.747, y:-7.746
navigation-7 [INFO] [178388872.737399274] [navigation]: NAVIGATION STATUS | state=WAYPOINT_REACHED | goal=(13.75, -7.74) | queue=0 | Reached waypoint (13.747, -7.741)
navigation-7 [INFO] [178388872.738478965] [navigation]: NAVIGATION STATUS | state=END | goal=none | queue=0 | All queued waypoints completed
navigation-7 [INFO] [178388872.738862275] [navigation]: All queued goals succeeded
navigation-7 [INFO] [178388872.739373242] [navigation]: NAVIGATION STATUS | state=GOAL_REACHED | goal=none | queue=0 | Complete waypoint queue executed
navigation-7 [INFO] [178388872.739692372] [navigation]: Got new position
navigation-7 [INFO] [178388873.286495084] [navigation]: Got new position
```

- Queue returned to 0
- All queued goals succeeded

Result

Goals are executed sequentially and are no longer discarded.

Localization (in Sim)

Localization Prerequisites

The robot knows:

- Odometry (drifts)
- Map
- LaserScan (noisy)

But not its location

So we need:

- Initial Pose
- Next Pose based on Odom
- Correct based on LiDAR-Data and Map

Localization Logic

Basic Logic

- Initial Pose is given
- Odom is added to Pose (Covariance grows)
- LaserScan gets matched to map (Covariance declines)
(Search window only in proximity allowed)
- Pose set to LaserScan pose

Deeper Logic

- Covariance grows more with more movement
- Once Covariance grows too much, global search is started
- Global search is set the the new pose

LaserScan Matcher Logic

Basic Logic

- LaserScan converted to 2D coordinates in map
- Score assigned to poses in search window based on likelihood field
- Best score becomes new pose

Deeper Logic

- Three search stages (coarse, medium, fine)
- Each stage gets narrower/smaller
- Score assigned to poses based on:
 - closeness to odom estimation
 - closeness to the likelihood field (gaussian distribution)

Global Search Logic

Basic Logic

- Takes many random poses in map and scores them
- Best matched score gets set to new pose
- Covariances get set low again

Problems

- Score Logic gets drawn to walls
- Problem with global search
- Closest spot to walls always wins

Result

Fails almost everytime

Reason that initial pose has to be given

Localization Performance

Performance with initial pose

- Tracks consistently
- Sometimes problems with hard turns

Performance with global search

- First pose almost always wrong
- Does not recover due to proximity scoring

Possible Fixes and Future Work

- Switch to particle tracking
- Tweak scoring
- Increase covariance more with turns